

TEORÍA (8 puntos)**Apellidos y nombre:****Pregunta 1. Diferencia entre «condición» y «decisión»****Pregunta 2. Técnicas basadas en puntos de decisión.**Dado el siguiente código fuente escrito en un lenguaje de operadores lógicos **no** cortocircuitados:

```
if mes >= 1 and mes <= 12
    then mostrarCalendario(mes);
end if;
```

¿Cuál es el mayor porcentaje de cobertura de condiciones múltiple que se podría conseguir?
Justifica tu respuesta.

Pregunta 3. Escribe las etiquetas equivalentes en JUnit 4 a los elementos que se presentan a continuación de JUnit 5:

JUnit 5	JUnit 4
@BeforeAll	
@AfterEach	
@Disable	
@AfterAll	
@Test	
@BeforeEach	
assertThrows(Exception.class, () -> {...});	
assertTimeout(ofSeconds(1), () -> {...});	

Pregunta 4. ¿Quién es el responsable de ejecutar las pruebas?

Pregunta 5. Dada una estimación previa del esfuerzo en nº de horas requerido por un proceso de pruebas, describe brevemente dos técnicas que permitan repartir ese número de horas entre distintas actividades o componentes:

Pregunta 6. ¿En qué consiste un *script* de pruebas?

Pregunta 7. Asumiendo que estás realizando el análisis de riesgos de un producto, ¿qué factores debemos tener en cuenta para asignar una clase de riesgo A, B o C a un componente de una característica de calidad cuando asignamos la clase de riesgo a los distintos objetivos de prueba asociados con esa característica de calidad?

Pregunta 8. ¿Qué se puede automatizar cuando se realizan pruebas de usabilidad? Nombra al menos dos herramientas que conozcas.

Pregunta 9. Describe los distintos tipos de causas de un defecto que sería conveniente anotar junto al defecto en una herramienta de gestión de defectos.

Pregunta 10. Enumera al menos tres características que se deberían comprobar durante la evaluación estática del documento correspondiente a un catálogo de requisitos.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

El *random bowling* es una variedad de bolos americanos en la que el número de lanzamientos de cada jugador en una partida se define de forma aleatoria antes de comenzar el juego. En el caso de algún jugador con mala suerte, este número de lanzamientos puede incluso ser igual a 0.

Por lo demás, el sistema de puntuación es el mismo que en los bolos americanos: cada bolo derribado en cada lanzamiento otorga un punto; si en un lanzamiento se derriban los 10 bolos (pleno), el número de bolos derribados en los dos siguientes lanzamientos se añaden a la puntuación de forma adicional; y si en un determinado juego, el jugador derriba el total de los 10 bolos en los dos lanzamientos del juego (semipleno), el número de bolos derribados en el siguiente lanzamiento se le añadirá a la puntuación de forma adicional.

En todo caso, como el número de lanzamientos se define al inicio de la partida, cabe la posibilidad de que un jugador que haya hecho pleno o semipleno en sus últimos lanzamientos no pueda sumar las bonificaciones correspondientes a las siguientes tiradas, al no poder realizar más lanzamientos que los que se le han otorgado inicialmente.

El método `calcularPuntuacion()` de la clase `RandomBowling` calcula la puntuación obtenida por un jugador en una partida de *random bowling*. A diferencia de la solución del kata «Bowling Game» que practicamos en el *coding dojo* del 14 de abril, en este caso, se ha almacenado el número de bolos derribados por el jugador en cada lanzamiento en el atributo `bolos`. Otros métodos de la clase `RandomBowling` se encargan de inicializarlo antes de invocar al método `calcularPuntuacion()`.

Cabe mencionar que en el método `calcularPuntuacion()`, el acceso al vector `bolos` se hace indirectamente a través del método `bolos()`, que devuelve 0 en el caso de intentar acceder a una componente inexistente del vector. Esto simplifica el problema de calcular la puntuación en el caso de jugadores que en los últimos lanzamientos han conseguido un pleno o sempleno y que tendrían que ser bonificados con las puntuaciones de jugadas posteriores y que no se han realizado.

```
public class RandomBowling {
    private int[] bolos;

    public int bolos(int lanzamiento) {
        if (lanzamiento < this.bolos.length) {
            return bolos[lanzamiento];
        } else {
            return 0;
        }
    }

    public int calcularPuntuacion() {
        A int puntuacion = 0;
        A int lanzamiento = 0;
        while ( B lanzamiento < bolos.length ) {
            if ( C bolos(lanzamiento) == 10 ) {
                D puntuacion += 10 + bolos(lanzamiento + 1) + bolos(lanzamiento + 2);
                lanzamiento++;
            } else if ( E bolos(lanzamiento) + bolos(lanzamiento + 1) == 10 ) {
                F puntuacion += 10 + bolos(lanzamiento + 2);
                lanzamiento += 2;
            } else {
                G puntuacion += bolos(lanzamiento) + bolos(lanzamiento + 1);
                lanzamiento += 2;
            } H
        }
        I return puntuacion;
    }

    ...
}
```

Se pide:

1. Obtener en primer lugar el grafo de flujo asociado al método `calcularPuntuacion()`, de acuerdo con la correspondencia entre instrucciones/condiciones y nodos que se indica en el propio código (0,25 puntos).
2. Aplicar la técnica de pruebas de caminos con **profundidad 3** para obtener las situaciones de prueba y el conjunto de caminos completos correspondiente (0,75 puntos).
3. Finalmente, establecer los casos de prueba correspondientes a los caminos obtenidos, incluyendo el valores del atributo de tipo vector `bolos` y su valor devuelto (1 punto).